

BraveType

FONT MANAGER TOOL — COMPLETE TECHNICAL ARCHITECTURE & SPECIFICATION MANUAL

Exhaustive Knowledge Transfer for Lead Software Architects & Core Engineers

Original Developer & Author: Brave Studios

Application Version: 1.0.0 (Production Stable)

Target OS: Windows 10 / Windows 11 (x64 / ARM64)

Tech Stack: Electron 33, React 18, TypeScript 5, Vite 6, Tailwind CSS 4, fontkit

Generated Date: August 2026

Table of Contents

1. Project Overview
2. Overall Architecture
3. Project Structure
4. Frontend
5. Backend
6. Database & Persistent Storage
7. APIs & IPC Communication Protocols
8. Authentication & Security Boundary
9. Important Classes & Core Functions
10. Complete Feature Walkthrough
11. Third-Party Dependencies Audit
12. Configuration & Build System
13. Current Progress & Technical Debt
14. Development Guidelines & Best Practices
15. Critical Files & Risk Map
16. Development Roadmap

1. Project Overview

BraveType - Font Manager Tool is a desktop font management application designed specifically for graphic designers, UI/UX engineers, branding agencies, and typographers operating on Windows.

1.1 Purpose

On Windows systems, typeface collections installed across `C:\Windows\Fonts` and `%LOCALAPPDATA%\Microsoft\Windows\Fonts` frequently grow to thousands of `.ttf`, `.otf`, `.woff`, `.woff2`, and `.ttc` font files. Stock OS tools fail to provide real-time preview customization, specimen waterfall scaling, or grouping into project folders. BraveType provides a high-performance offline font manager capable of cold-booting in under 50ms, rendering specimen previews smoothly, and managing custom font collections with zero cloud dependencies.

1.2 Core Goals

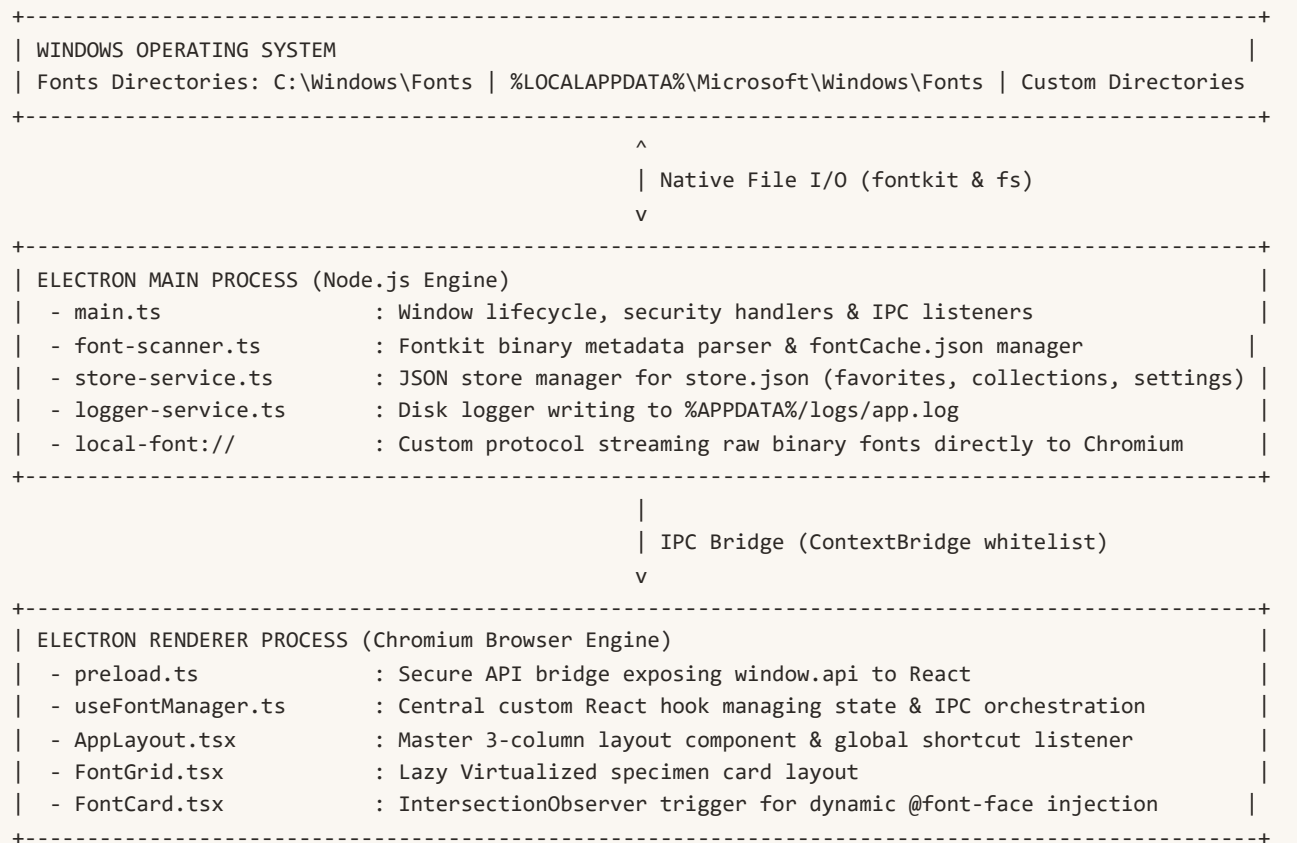
- **Instant Cold Boot (<50ms):** Bypasses slow OS directory re-scans on boot using a dual-stage metadata cache file (`fontCache.json`).
- **Zero-Jank Viewport Specimen Rendering:** Utilizes browser `IntersectionObserver` API and dynamic CSS `@font-face` injection to load font binaries strictly when visible on screen.
- **Paper Minimal Design System:** Implements a visual theme built around warm neutrals (`#FAF7F2`), crisp cards (`#FFFFFF`), charcoal typography (`#2C2825`), and terracotta orange accents (`#E86A33`).
- **100% Offline & Private:** Zero telemetry, zero analytics, zero external network requests.

1.3 Features At A Glance

- **Fast System Directory Scanner:** Background thread scanner that indexes installed fonts across system, user, and custom drive directories without freezing the UI thread.
- **9 Type Specimen Layout Presets:** Preset modes for *Sentence*, *Paragraph*, *Alphabet*, *Numbers*, *Heading*, *Logo*, *Poster*, *Button*, and *Business Card*.
- **Dynamic `local-font://` Protocol:** Custom streaming protocol that serves binary fonts directly from disk to Chromium without Base64 memory overhead.
- **Manual Collection Folders:** Create, rename, duplicate, alphabetize, and drag-and-drop font families into custom workspace folders.
- **Modal Specimen Inspector:** Tabbed full-screen modal offering Waterfall scale views (14px–80px), style matrices, character glyph grids, and technical table details.
- **Developer Mode:** Exposes PostScript names, OpenType table structures, glyph counts, Unicode coverage, and instant CSS `font-family` snippet export.
- **Synthesizer Click Feedback:** Embedded Web Audio API audio node synthesizer that plays mechanical keyboard click sounds during splash initialization.
- **JSON Store Backup:** One-click JSON export/import of user favorites, custom collections, and application settings.

2. Overall Architecture

BraveType utilizes Electron's multi-process model, cleanly separating low-level OS filesystem tasks from the Chromium browser rendering engine.



2.1 Key Design Decisions

- **Dual-Process Isolation:** Node.js filesystem I/O and heavy binary parsing are strictly contained inside the Main process. The Renderer process runs with `nodeIntegration: false` and `contextIsolation: true`.
- **Custom Dynamic Protocol Stream (`local-font://`):** Storing thousands of binary fonts as Base64 strings inside DOM elements consumes multiple gigabytes of RAM. Serving them via a custom protocol streams font files on-demand directly from disk.
- **IntersectionObserver Lazy Loading:** Font cards defer dynamic `@font-face` injection until they scroll within 200px of the viewport, keeping memory usage constant regardless of font count.

3. Project Structure

```

Fontbrave/
├── build/                # App packaging icon assets (icon.png, icon.svg)
├── dist/                 # Compiled Vite frontend distribution output
├── dist-electron/        # Compiled Main process JS (main.js, preload.js)
├── electron/             # Main Process TypeScript Source Code
│   ├── font-scanner.ts   # OS font directory scanner & fontkit binary parser
│   ├── logger-service.ts # File logger writing to %APPDATA%/logs/app.log
│   ├── main.ts           # Electron app entry, custom protocol, & IPC handlers
│   ├── preload.ts        # Secure ContextBridge whitelist script
│   ├── store-service.ts  # Persistent storage manager for store.json
│   └── update-service.ts  # Application version update checker service
├── scripts/              # Packaging and release automation scripts
│   ├── fix-wincodesign.js # Fixes winCodeSign cache symlink issues on Windows
│   ├── generate-icon.js   # Programmatically generates 512x512 PNG & SVG icons
│   ├── prepare-release.js # Creates version manifests & SHA256 checksums
│   └── prepare-wincodesign.js # Extracts 7z signing tool archives
├── src/                  # Frontend React Source Code
│   ├── components/
│   │   ├── Branding/BrandLogo.tsx # Header brand icon & title
│   │   ├── CenterArea/
│   │   │   ├── FontCard.tsx       # Individual specimen preview card
│   │   │   ├── FontGrid.tsx       # Grid/List layout container
│   │   │   ├── HeaderBar.tsx      # Search, font size slider, preview text & mode pills
│   │   │   └── PreviewModeSelector.tsx # 9 Specimen preview mode selector pills
│   │   ├── Layout/AppLayout.tsx   # Master 3-column container & keyboard hotkey host
│   │   ├── Modals/
│   │   │   ├── FontDetailModal.tsx # Full spec sheet, character map, & waterfall modal
│   │   │   └── SettingsModal.tsx   # Preferences, custom paths, & backup modal
│   │   ├── RightPanel/
│   │   │   ├── FontInspector.tsx   # Inspector side panel
│   │   │   ├── GlyphGrid.tsx       # Character map unicode grid
│   │   │   ├── MetadataTable.tsx   # Font technical specs & CSS generator
│   │   │   └── StyleSelector.tsx    # Font weight/style dropdown picker
│   │   ├── Sidebar/
│   │   │   ├── CollectionFolderItem.tsx # Manual folder item supporting drag & drop
│   │   │   ├── CreateFolderModal.tsx   # New collection folder modal
│   │   │   └── Sidebar.tsx             # Navigation sidebar
│   │   └── Splash/SplashScreen.tsx     # Typing splash screen & Web Audio synthesizer
│   ├── hooks/useFontManager.ts         # Central custom React state hook
│   ├── types/                          # TypeScript type definitions (font.ts, store.ts)
│   ├── utils/                          # Helper modules (audioUtils, fontLoader, glyphUtils)
│   ├── App.tsx                         # Root App component
│   └── index.css                       # Tailwind CSS v4 design tokens & base rules
├── index.html                         # HTML root template
├── package.json                      # Build dependencies & electron-builder settings
├── tailwind.config.js                # Tailwind CSS theme extension rules
└── vite.config.ts                    # Vite bundler configuration

```

4. Frontend

4.1 3-Column Layout

The UI is structured into three primary structural regions inside `AppLayout.tsx`:

1. **Left Sidebar (256px):** Contains search input, category filters (*All Fonts, Favorites, Recently Used*), custom user folders, manual folder creation/sorting controls, and Settings trigger.
2. **Center Area (Flex-1):** Top bar containing font size slider (14px–72px), layout view toggle (Grid vs List), custom preview text input, 9 specimen preview preset pills, and scrollable font card area.
3. **Right Inspector Panel (320px–384px):** Selected font family header, style dropdown selector, specimen text preview, character map glyph grid, metadata table, and show in Explorer trigger.

4.2 State Management

Application state is managed using a single React hook: `useFontManager()` in `src/hooks/useFontManager.ts`. It handles font lists, active query strings, category filters, font selection, preview text, preview modes, favorites arrays, collection arrays, recent font history, and application settings. Any mutation to persistent state (favorites, collections, settings) automatically sends IPC updates to persist changes into disk storage (`store.json`).

4.3 Design System & Styling

Built with Tailwind CSS v4 using a Paper Minimal aesthetic system defined in `src/index.css`:

Token Name	Color Hex	Visual Purpose
<code>paper-bg</code>	<code>#FAF7F2</code>	Warm off-white background canvas
<code>paper-cream</code>	<code>#F7F3EC</code>	Sidebar & panel container cards
<code>paper-card</code>	<code>FFFFFF</code>	Elevated card containers & inputs
<code>paper-border</code>	<code>#EFE8DD</code>	Subtle divider borders
<code>accent-orange</code>	<code>#E86A33</code>	Terracotta primary interactive accent
<code>charcoal-main</code>	<code>#2C2825</code>	Primary typography headings & body
<code>charcoal-muted</code>	<code>#78716C</code>	Secondary metadata descriptions

5. Backend Architecture

The main process consists of specialized service modules inside `electron/`:

- **FontScannerService** (`electron/font-scanner.ts`): Iterates through OS font directories. Checks file modification timestamps (`mtimeMs`) against `fontCache.json` to bypass re-parsing unchanged files. Opens

missing files with `fontkit` to extract OpenType parameters, subfamily names, weight classes, width classes, variable variation axes, and glyph counts.

- **StoreService** (`electron/store-service.ts`): Manages read/write operations for `%APPDATA%/bravetype-font-manager/store.json`. Sanitizes input payload shapes, provides default collection bootstrapping, and implements store reset routines.
- **LoggerService** (`electron/logger-service.ts`): Writes timestamped diagnostic entries, system warnings, and error stack traces to `%APPDATA%/bravetype-font-manager/logs/app.log`.
- **UpdateService** (`electron/update-service.ts`): Provides release version state routines.

6. Database & Storage Schemas

BraveType stores application data in JSON files within the Windows `%APPDATA%\bravetype-font-manager\` directory:

6.1 `fontCache.json` Schema

```
{
  "version": "1.0.0",
  "lastScanned": 1770281720000,
  "families": [
    {
      "id": "roboto",
      "familyName": "Roboto",
      "category": "Sans-serif",
      "isVariable": false,
      "postScriptNames": ["Roboto-Regular", "Roboto-Bold"],
      "styles": [
        {
          "id": "roboto-regular-400",
          "name": "Regular",
          "weight": 400,
          "weightName": "Regular",
          "width": "Normal",
          "postScriptName": "Roboto-Regular",
          "filePath": "C:\\Windows\\Fonts\\roboto.ttf",
          "fileName": "roboto.ttf",
          "fileSize": 168200,
          "fileSizeFormatted": "164.3 KB",
          "mtimeMs": 1700000000000,
          "format": "TrueType",
          "isVariable": false,
          "glyphCount": 1294
        }
      ],
      "sampleStyle": { /* FontStyle Object */ }
    }
  ]
}
```

6.2 `store.json` Schema

```
{
  "favorites": ["roboto", "segoe-ui"],
  "collections": [
    {
      "id": "col-branding",
      "name": "Branding",
      "fontIds": ["roboto"],
      "createdAt": 1770281720000
    }
  ],
  "recentFonts": ["roboto"],
  "settings": {
    "previewSize": 28,
    "viewMode": "grid",
    "defaultPreviewText": "The quick brown fox jumps over the lazy dog",
    "language": "English",
    "customFontDirs": ["D:\\CustomFonts"],
    "enableStartupSound": true,
    "developerMode": false,
    "launchAtStartup": false
  }
}
```

7. APIs & IPC Protocols

7.1 IPC Endpoint Whitelist

IPC Channel Name	Purpose	Request Payload	Return Output Payload
<code>font:get-cached</code>	Reads cached metadata instantly	None	<code>ScannedFamily[]</code>
<code>font:scan</code>	Triggers background font directory scan	<code>customDirs?: string[]</code>	<code>ScannedFamily[]</code>
<code>font:scan-progress</code>	Broadcast event for scan progress updates	None	<code>{ count, total, statusText, currentFamily }</code>
<code>store:get</code>	Reads <code>store.json</code> data	None	<code>StoreData</code>
<code>store:save</code>	Saves partial store updates	<code>Partial<StoreData></code>	<code>StoreData</code>
<code>store:reset</code>	Resets store to factory defaults	None	<code>StoreData</code>
<code>system:open-location</code>	Reveals font file path in Explorer	<code>filePath: string</code>	<code>boolean</code>
<code>system:copy-clipboard</code>	Copies text string to Windows clipboard	<code>text: string</code>	<code>boolean</code>
<code>system:export-backup</code>	Opens Save Dialog to export backup JSON	None	<code>{ success: boolean, filePath?: string }</code>
<code>system:import-backup</code>	Opens Open Dialog to import backup JSON	None	<code>{ success: boolean, error?: string }</code>
<code>system:set-launch-startup</code>	Configures Windows startup launch registry	<code>enabled: boolean</code>	<code>boolean</code>

7.2 Custom Protocol: `local-font://`

Registered as a privileged protocol in `electron/main.ts`. Converts incoming request URLs (e.g. `local-font://C:/Windows/Fonts/segoeui.ttf`) into normalized OS paths, verifies file existence, determines MIME header (`font/ttf`, `font/otf`, `font/woff`, `font/woff2`), and streams raw font bytes directly to Chromium.

8. Authentication & Security Model

BraveType runs as an offline local tool enforcing sandboxed isolation boundaries:

- **No Node Integration in UI:** `nodeIntegration: false` and `contextIsolation: true` prevent untrusted code from invoking Node commands.
- **ContextBridge Whitelist:** All Main-Renderer IPC interfaces pass strictly through the secure `window.api` bridge defined in `electron/preload.ts`.
- **Window Handler Blocking:** `setWindowOpenHandler(() => ({ action: 'deny' }))` prevents external popup windows from opening.
- **Navigation Guard:** Blocks window navigation outside local development URLs or local static `file://` templates.
- **Path Normalization:** `local-font://` protocol normalizes path slashes and checks file existence to prevent directory traversal exploits.

9. Important Classes & Core Functions

9.1 `FontScannerService` (`electron/font-scanner.ts`)

Discovers system fonts, parses binary files with `fontkit`, and manages `fontCache.json`. Implements time-based delta checking (`mtimeMs`) to bypass parsing unmodified font files.

9.2 `useFontManager()` (`src/hooks/useFontManager.ts`)

Central React hook managing application state. Computes filtered font lists using `useMemo`, handles selection states, and auto-persists updates to disk via `window.api.saveStore()`.

9.3 `ensureFontLoaded(style: FontStyle)` (`src/utils/fontLoader.ts`)

Injects dynamic CSS `@font-face` rules pointing to `local-font://` URLs into `document.head`. Keeps a `Set<string>` index to prevent duplicate style injections.

9.4 `playKeyboardClick(enabled: boolean)` (`src/utils/audioUtils.ts`)

Uses Web Audio API nodes (highpass filter + noise buffer burst + triangle wave oscillator) to generate real-time mechanical keyboard click sound effects during splash initialization.

10. Complete Feature Walkthrough

10.1 App Startup Sequence

1. User executes application binary.
2. Main process opens window and registers `local-font://` protocol.

3. `SplashScreen.tsx` renders typing animation for "Brave Studios" accompanied by synthesized key click sound effects.
4. `useFontManager` calls `window.api.getStore()` and `window.api.getCachedFonts()`.
5. Cached fonts display in UI within ~50ms; splash screen smoothly fades out once initial scan completes.

10.2 Specimen Rendering & Lazy Injection

1. User scrolls through the font list in `FontGrid.tsx`.
2. An `IntersectionObserver` in `FontCard.tsx` detects when a card enters the viewport.
3. Card calls `ensureFontLoaded()`, injecting a custom `@font-face` rule into document head.
4. Chromium issues a request to `local-font://`, streaming raw binary bytes into rendering engine.

10.3 Drag & Drop Collection Management

1. User creates a collection folder in `Sidebar.tsx`.
2. User initiates drag on a font card (`onDragStart` sets font ID in `dataTransfer`).
3. User drops card over `CollectionFolderItem.tsx` (`onDrop`).
4. Font ID is added to collection's `fontIds` list and saved to `store.json`.

11. Third-Party Dependencies Audit

Library Name	Core Functionality	Risk / Impact if Removed
<code>fontkit</code>	Binary parser for OpenType/TrueType tables	Font scanning, weight parsing, and glyph counting fail completely.
<code>electron</code>	Desktop application runtime container	Desktop OS integration and local file execution fail.
<code>electron-builder</code>	Packager for NSIS installers & Portable EXEs	Cannot package standalone Windows installer binaries.
<code>lucide-react</code>	UI iconography suite	Visual interface icons fail to render.
<code>clsx</code> / <code>tailwind-merge</code>	Utility class string merging	Dynamic Tailwind class application breaks.
<code>framer-motion</code>	Component micro-animations	Modal layout transitions degrade.

12. Configuration & Build System

- **Vite Configuration** (`vite.config.ts`): Configures React plugin, dev server on port 5173, path alias `@ -> ./src`, and `vite-plugin-electron` main/preload build entries with `fontkit` marked as external.

- **Packaging Configuration (`package.json`):** Configures `electron-builder` target options for NSIS installers and Portable executables, referencing output directory `Release/`.
- **Build Commands:**
 - `npm run dev` : Starts Vite dev server + Electron with HMR live reloads.
 - `npm run build` : Compiles TypeScript frontend and backend modules.
 - `npm run dist` : Generates installer binaries in `Release/` directory.
 - `npm run release` : Prepares release folder with SHA256 checksums and version manifests.

13. Current Progress & Roadmap

13.1 Completed Work

Fully functional offline font management platform with fast caching, viewport lazy loading, 9 preview modes, manual collections, character maps, developer mode, and backup import/export utilities.

13.2 Architecture Reserved Placeholders

Found in `src/types/font.ts` for future V2 releases:

- `aiCategory?: string;`
- `aiTags?: string[];`
- `similarFontIds?: string[];`
- `fontPairings?: string[];`

13.3 Development Roadmap

1. **V2 Font Pairing Engine:** Implement offline font pairing recommendation engine using `fontPairings` placeholders.
2. **Variable Axis Controls:** Interactive real-time sliders for OpenType variation axes (`wght` , `width` , `slnt` , `opsz`).
3. **Multi-Font Comparison Split:** Pin and compare multiple typeface specimens side-by-side.

14. Development Guidelines & Best Practices

1. **Strict TypeScript Interfaces:** Keep types synchronized across `src/types/font.ts` and `src/types/store.ts`.
2. **Standardized IPC Channel Names:** Follow `domain:action` format (e.g. `font:scan` , `store:save` , `system:copy-clipboard`).
3. **No Direct File Calls in UI:** Interact with the host OS strictly via the `window.api` whitelist bridge.
4. **Clean Observer Cleanup:** Disconnect `IntersectionObserver` instances in React cleanup hooks to prevent memory leaks.

15. Critical Files Summary

- `electron/main.ts` : Application lifecycle, window creation, custom protocol registration, and IPC handlers.
- `electron/font-scanner.ts` : Binary file parsing engine and `fontCache.json` manager.
- `electron/preload.ts` : Secure context bridge exposing `window.api` .
- `src/hooks/useFontManager.ts` : Central React state hook managing application logic.
- `src/utils/fontLoader.ts` : Dynamic CSS `@font-face` injection module.
- `src/components/Layout/AppLayout.tsx` : Master 3-column layout container and global hotkey host.

16. Conclusion

This technical specification provides the complete knowledge transfer required to maintain, extend, and architect future versions of **BraveType - Font Manager Tool**. Happy coding!